

A3

Equipes de no máximo 3 alunos:

Preencher a planilha com os dados da equipe:

<https://docs.google.com/spreadsheets/d/1ndtMmPr9bxL5L0QUjN-HKsymhTc080OCIKVSOfNAlhY/edit?usp=sharing>

Obrigatória a presença de toda a equipe para possível apresentação de algumas equipes ao professor. Caso o aluno não compareça ficará sem nota na atividade;

Nesta atividade, você deverá aplicar os conceitos estudados na disciplina de **Sistemas Distribuídos**, por meio do desenvolvimento de uma aplicação prática utilizando **Java + Spring Boot + Front End**.

O foco desta atividade não é apenas o desenvolvimento de uma API, mas também a aplicação de conceitos relacionados à **arquitetura distribuída, separação de serviços e comunicação entre sistemas**.

Objetivo

Desenvolver uma aplicação baseada em **microserviços simples**, com comunicação via **API REST**, implementando operações de cadastro (CRUD) e relacionamento entre entidades.

Descrição da Atividade

Você deverá desenvolver uma solução composta por **no mínimo 3 microserviços**, considerando:

- Cada serviço deve possuir responsabilidades bem definidas;
- Os domínios devem possuir relação entre si (ex: Cidade/UF, Produto/Categoria, Pedido/Cliente);
- Os serviços devem ser independentes (execução separada);

- Deve haver comunicação entre os microserviços via REST (obrigatório);

Requisitos Funcionais

- Implementar ao menos **3 entidades (cadastros)**;
- As entidades devem possuir **relacionamento entre si**;
- Implementar operações de:
 - Cadastro (POST)
 - Consulta por ID e listagem geral (GET)
 - Atualização (PUT)
 - Remoção (DELETE)

Requisitos Técnicos

- Utilizar **Java com Spring Boot**;
- Utilizar **API REST** para comunicação;
- Utilizar **MySQL** como banco de dados;
- Aplicar boas práticas de desenvolvimento:
 - Separação em camadas (Controller, Service, Repository);
 - Organização do projeto;
 - Nomenclatura clara e adequada;

Integração entre Microserviços (Obrigatório)

- Pelo menos um microserviço deve consumir dados de outro;
- A comunicação deve ocorrer via chamada HTTP (REST);
- Exemplos:
 - Serviço de Pedido consultando dados de Cliente;
 - Serviço de Produto consultando Categoria;

Exemplos de Temas (Sugestões)

- Sistema de pedidos (Cliente, Pedido, Produto)

- Sistema acadêmico (Aluno, Curso, Matrícula)
- Sistema de biblioteca (Livro, Autor, Empréstimo)

(O tema é livre, desde que atenda aos requisitos)

FRONT-END

Desenvolva uma interface simples para consumo dos microserviços:

- A aplicação pode conter:
 - Uma tela principal (menu);
 - Telas de cadastro para cada entidade;
- Utilize a tecnologia de sua preferência (React, Vue, PHP, etc.);
- O front-end deve consumir as APIs desenvolvidas;

Diferenciais (Opcional – Bônus)

- Uso de DTOs;
- Tratamento de exceções;
- Documentação com Swagger/OpenAPI;
- Testes básicos;
- Integração com serviços externos;

O QUE VOCÊ DEVE ENTREGAR

- Repositório no GitHub contendo:
 - Código-fonte completo (backend e frontend);
 - Instruções para execução do projeto (README);
- Descrição da arquitetura:
 - Quais são os microserviços;
 - Como ocorre a comunicação entre eles;
- Vídeo demonstrando o funcionamento da aplicação (publicado no YouTube);
- Mais informações sobre a entrega serão repassadas nas próximas aulas;

CRITÉRIOS DE AVALIAÇÃO

- Implementação dos requisitos funcionais (20%)
- Uso correto de microserviços (20%)
- Organização e boas práticas de código (25%)
- Funcionamento geral da aplicação (25%)
- Front-end integrado (10%)

Penalizações Importantes

- Projetos sem front end funcional terão desconto significativo;
- Não utilização da stack definida implicará perda de pontos;

Observações Importantes

- O foco da atividade é demonstrar entendimento de **sistemas distribuídos**, e não apenas a implementação de CRUDs;
- A simplicidade da interface não será penalizada, desde que funcione corretamente;
- Valorize organização, clareza e boas práticas no desenvolvimento;